

MERMAID 11.14 • LATTICE THEME

# Every Mermaid diagram, one slide each.

*A reference gallery of the 26 diagram types Mermaid supports, rendered with the Lattice palette so you can see what each one looks like in production.*



# How this gallery is organized.

## Three groups, twenty-six diagrams

Stable diagrams render with full theme control. Experimental ones marked 🔥 may evolve in syntax and have limited theming surfaces.

15

### Stable

Flowchart, sequence, class, state, ER, journey, gantt, pie, quadrant, requirement, gitgraph, c4, mindmap, timeline, zenuml.

11

### Experimental 🔥

Sankey, xy chart, block, packet, kanban, architecture, radar, treemap, venn, ishikawa, treeView.

6

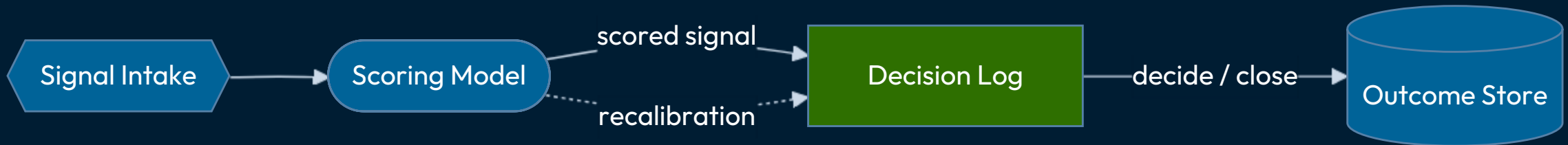
### Themeable

Flowchart, sequence, pie, state, class, journey have named themeVariables. Everything else inherits from the core six.

GROUP 01 · STABLE DIAGRAMS

# The fifteen well-supported types.

## Boxes and arrows. The most-used diagram type.

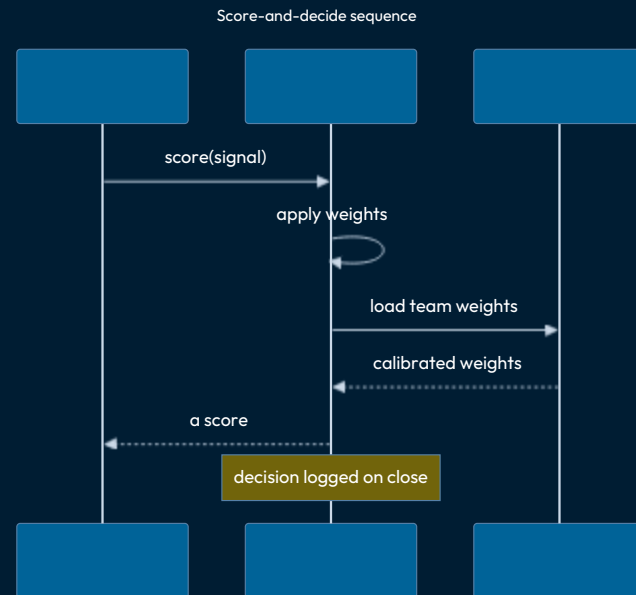


### KEY INSIGHT

Full theme support — node fills, borders, edge colors, cluster fills, edge labels all controllable via themeVariables.

## 02 · SEQUENCE DIAGRAM

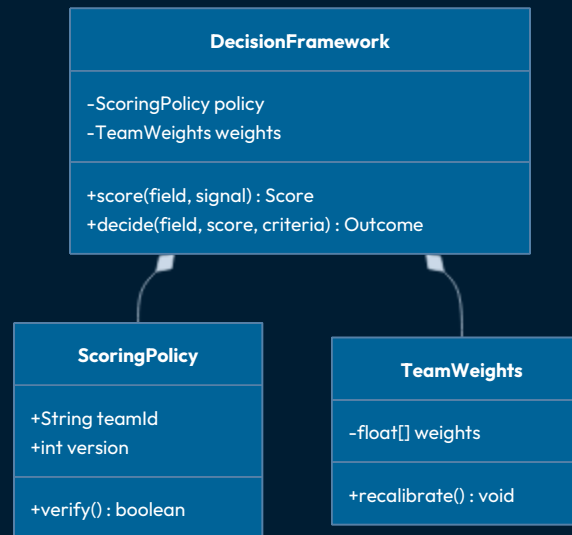
# Actors and signals over time.

**KEY INSIGHT**

Full theme support — actor backgrounds, signal colors, activation bars, note styling all themeable.

## 03 · CLASS DIAGRAM

# UML classes with attributes, methods, and inheritance.

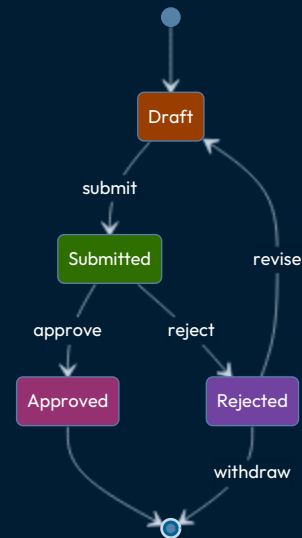


## KEY INSIGHT

Themeable — `classText` controls label color; node fills inherit from `primaryColor`.

## 04 · STATE DIAGRAM

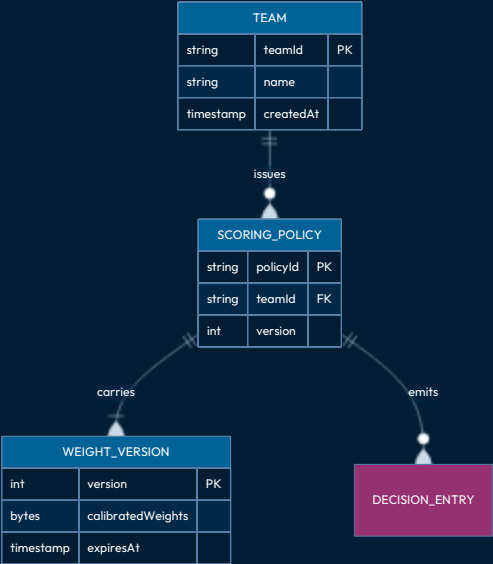
# A finite state machine.



## KEY INSIGHT

Themeable — `labelColor` and `altBackground` for composite states. State fills inherit from primary.

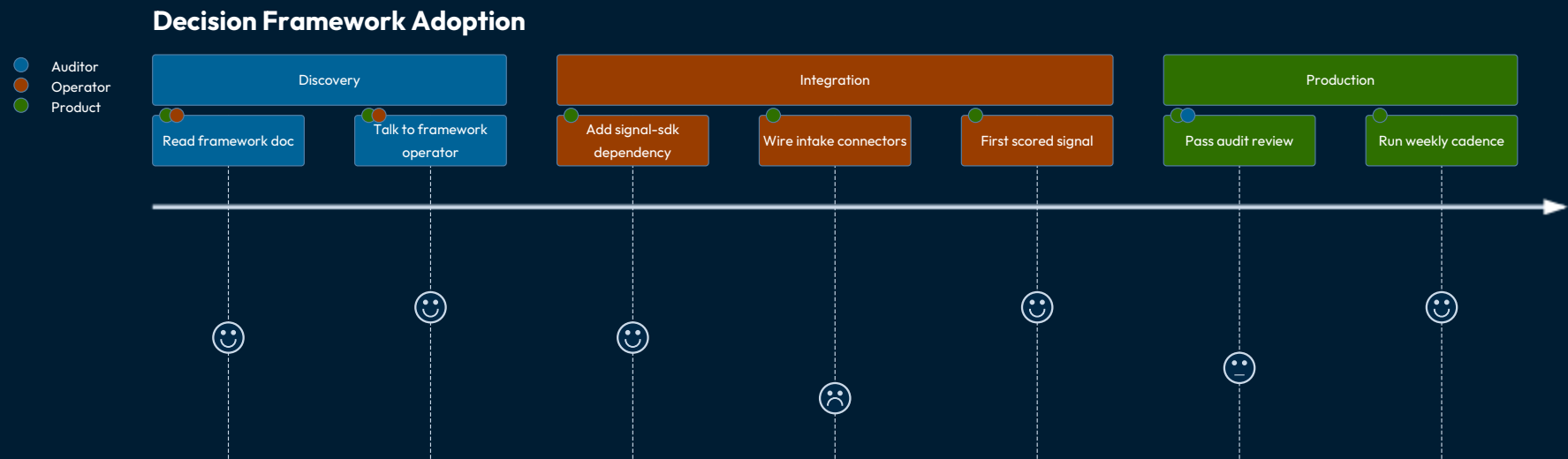
# Database tables and their relationships.



KEY INSIGHT

No named themeVariables — inherits border / text / fill from core variables.

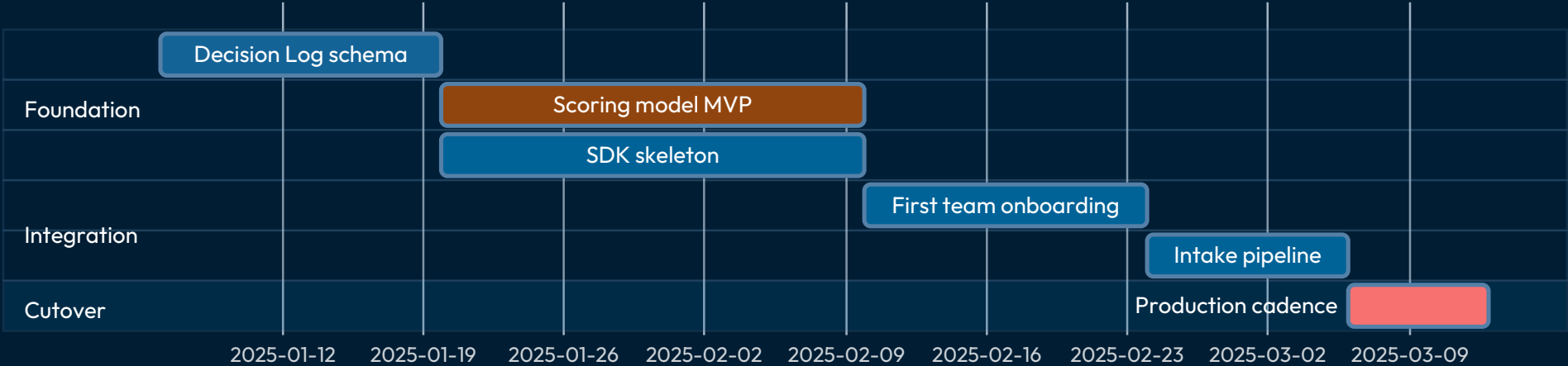
# Steps and the emotional state at each one.



KEY INSIGHT

Themeable — eight `fillType0..7` variables for the section coloring.

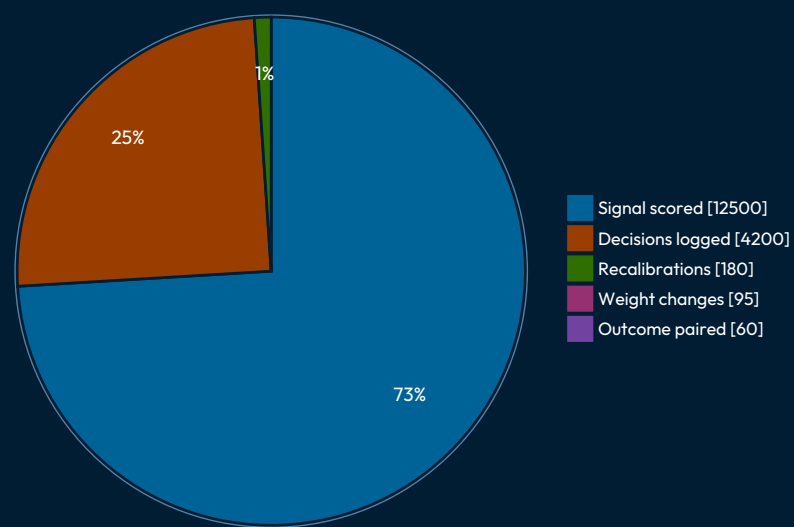
# Tasks across a timeline, with dependencies.



KEY INSIGHT

Lots of dedicated themeVariables — task bar, active task, done task, critical, today line all separately controllable.

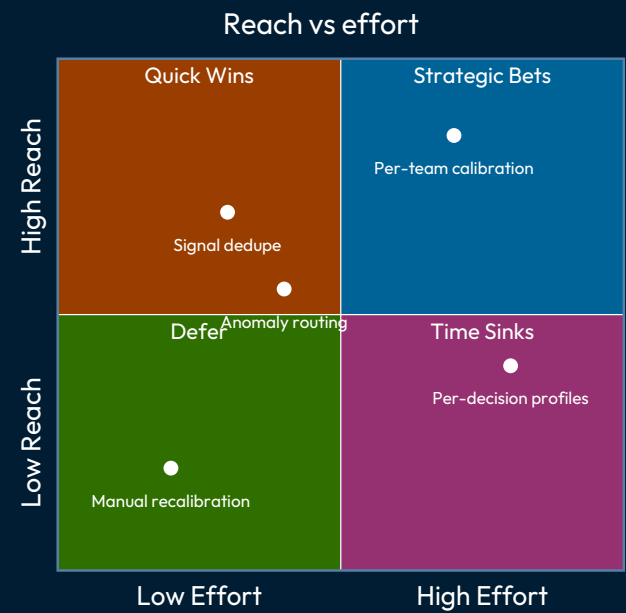
# Proportions of a whole.



KEY INSIGHT

Most theme-rich diagram — twelve `pie1..12` slot colors plus title, section, legend, stroke variables.

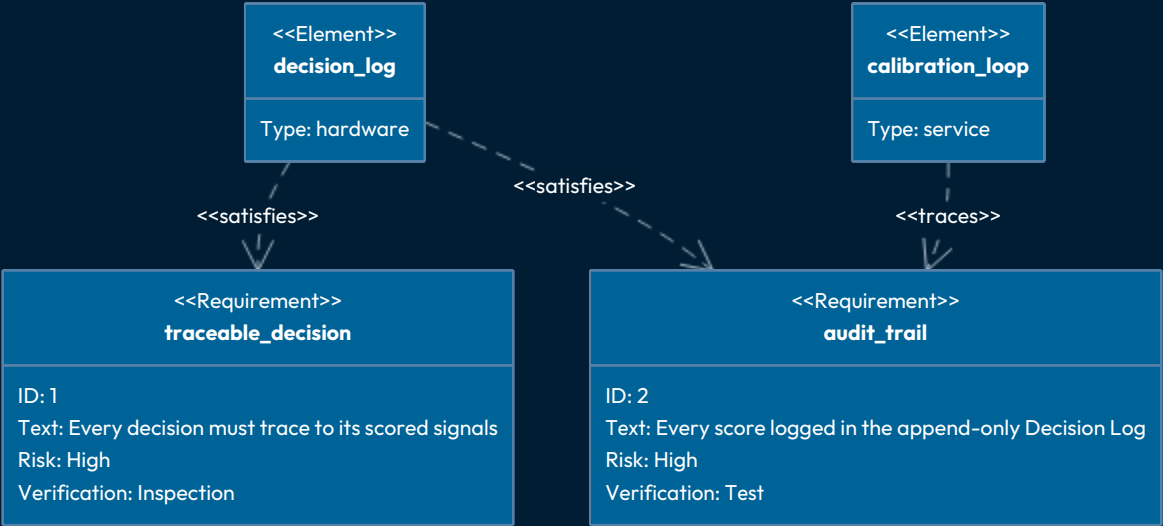
# Two-axis priority scatter.



KEY INSIGHT

Themeable — four quadrant fills and text colors, axis labels, point fill all controllable.

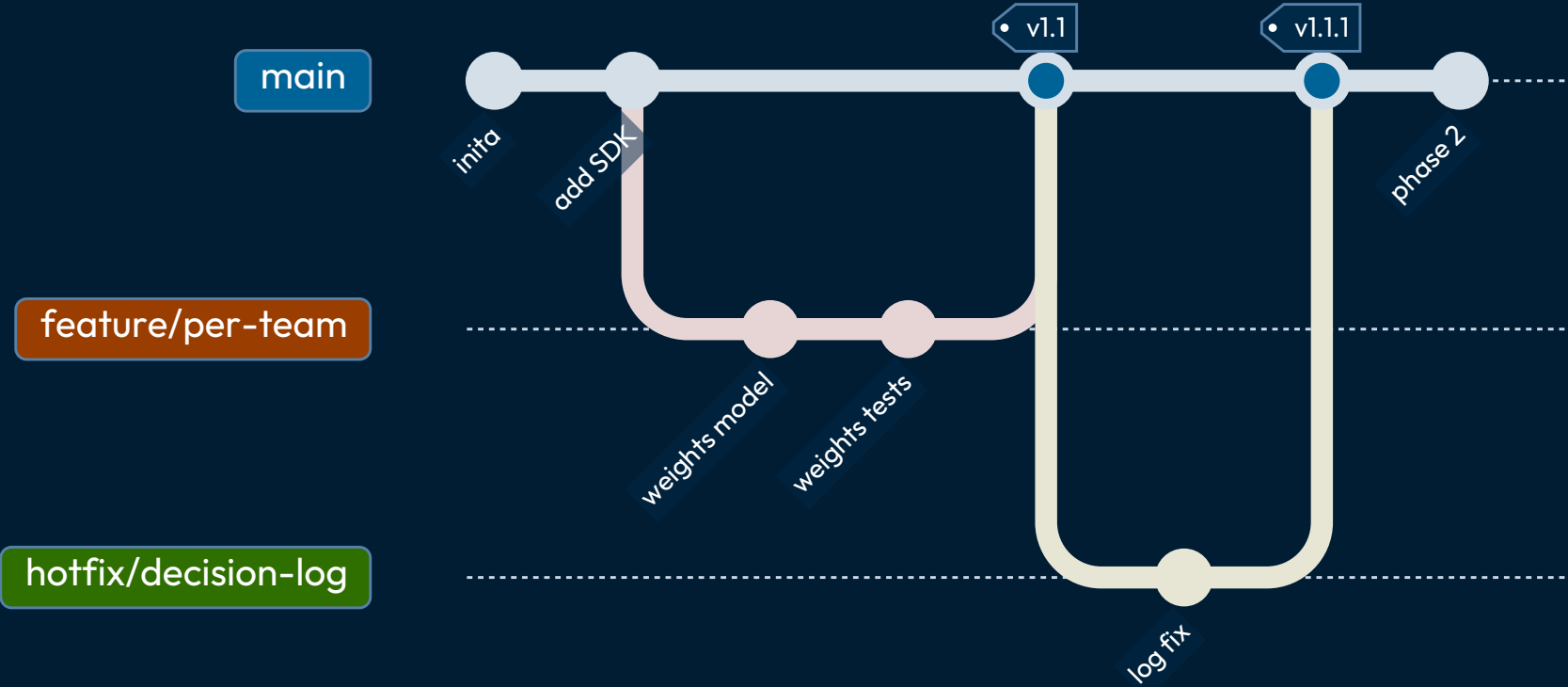
# SysML-style requirements with traceability.



KEY INSIGHT

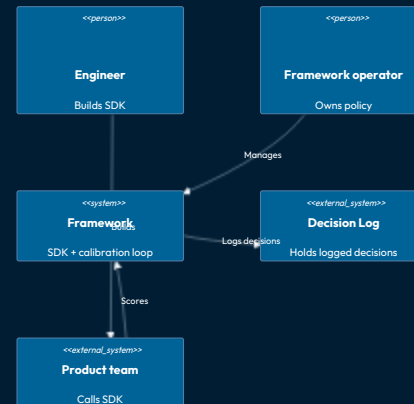
Inherits from core variables. No diagram-specific theme keys.

# Branching, merging, tags.



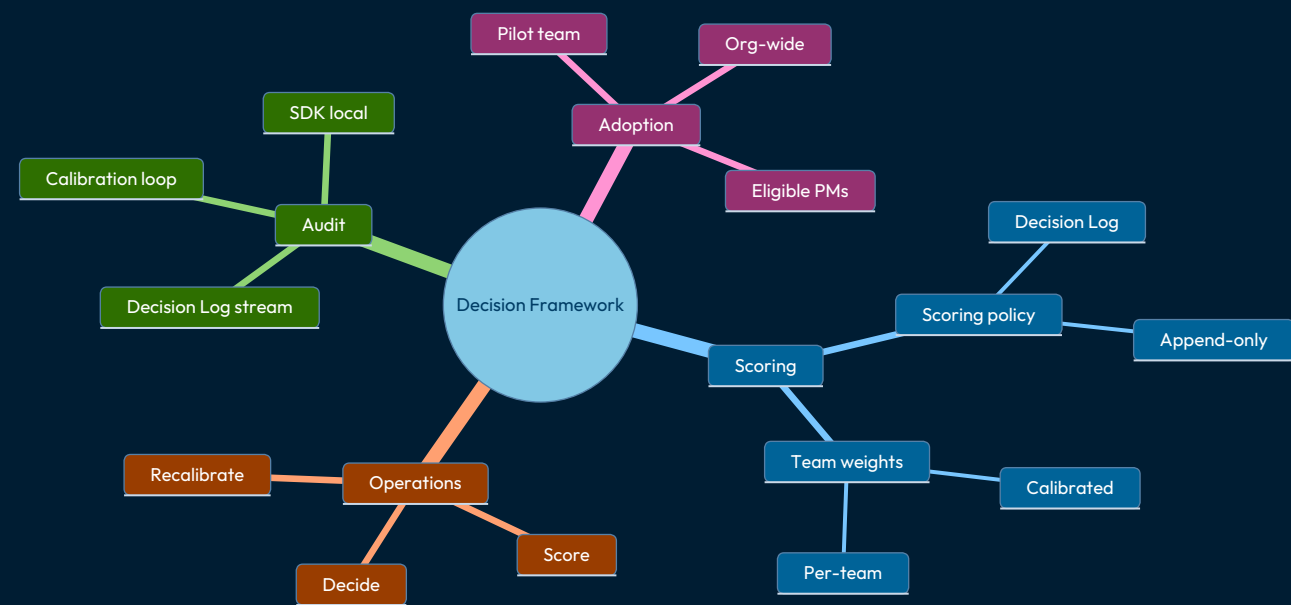
## 12 · C4 DIAGRAM

# Software architecture at four zoom levels.

**KEY INSIGHT**

Flagged work-in-progress in the docs — supported, but the team says so, and theme support is partial.

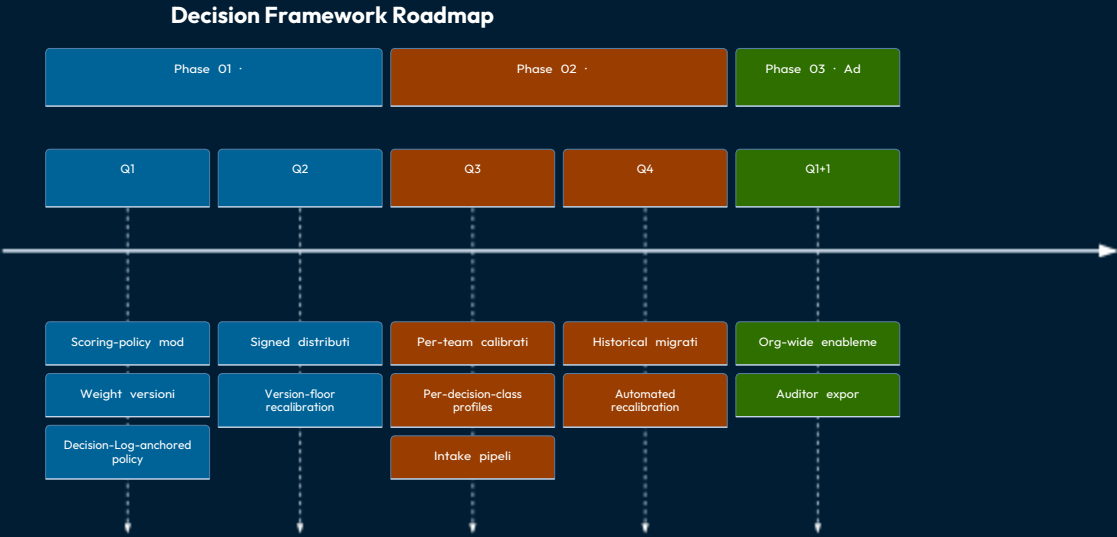
# Hierarchical brainstorm.



KEY INSIGHT

Inherits from core. Multiple shape syntaxes: `((round))`, `[square]`, `(rounded)`, `))cloud((`, `)hex(`.

# Events on a horizontal axis.



## KEY INSIGHT

Inherits from core. The section coloring uses the `cScale` palette.

## A simpler sequence-diagram dialect.

ZenUML is a Mermaid diagram type that emits HTML+SVG with Tailwind CSS classes for styling. The Mermaid CLI can produce the markup, but it does not bundle the @zenuml/core stylesheet — so static SVG export renders the structure without the visual styling. ZenUML works in browser contexts where the Tailwind utilities resolve.

For static-PDF pipelines like this one, prefer the standard `sequenceDiagram` (slide 5) — same conceptual model, fully supported by the Mermaid CLI, themable through `themeVariables`.

```
zenuml
title Order
Customer->>Order.create() {
  Order->>Inventory.check() {
    return available
  }
  return confirmed
}
```

### KEY INSIGHT

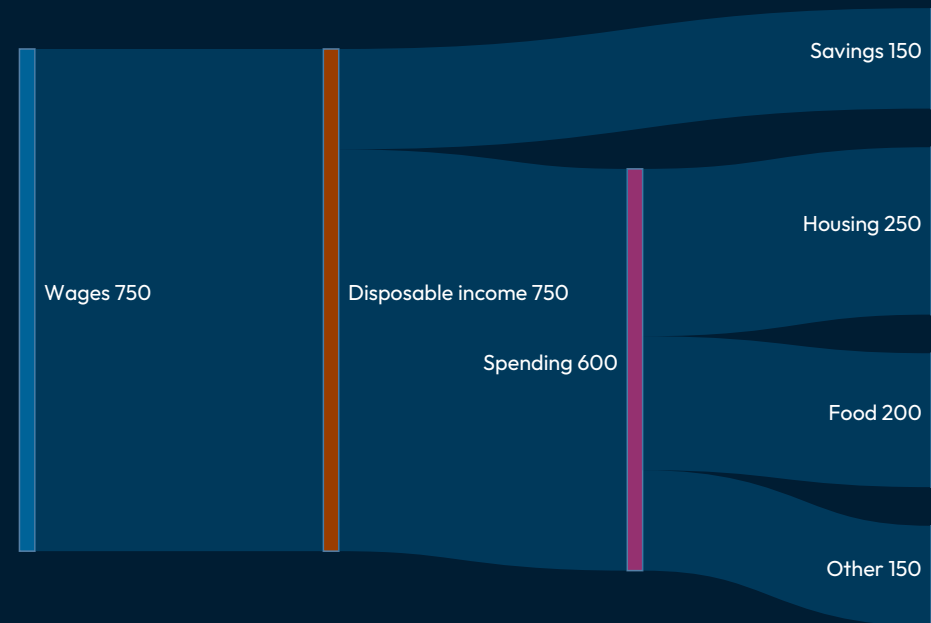
ZenUML uses HTML/Tailwind rendering rather than pure SVG. Use `sequenceDiagram` for static export contexts.

GROUP 02 · EXPERIMENTAL DIAGRAMS

The eleven new types marked 🔥.

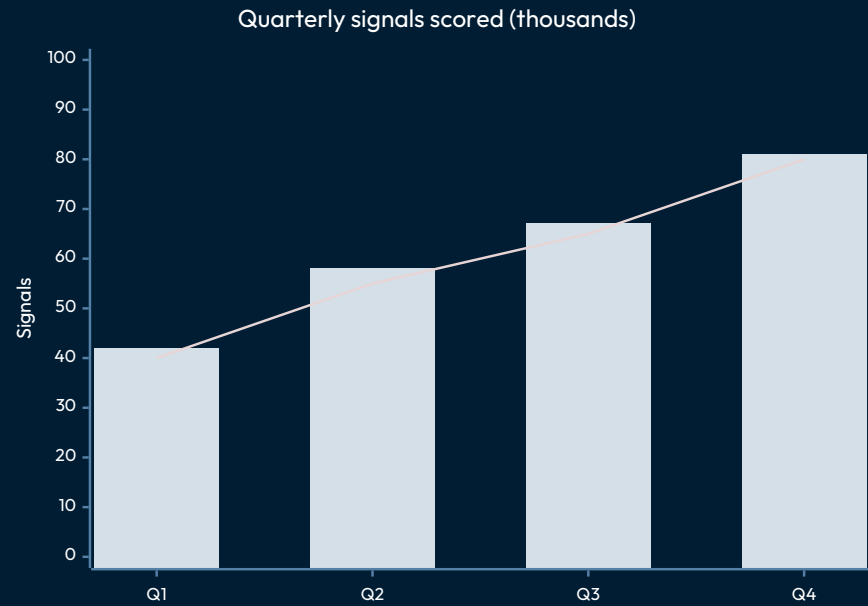
## 16 · SANKEY 🔥

## Flow volumes between nodes.

**KEY INSIGHT**

CSV-style input. Theme inheritance only — link colors are computed from node colors via the `linkColor` config.

# Bar and line chart on a numeric axis.

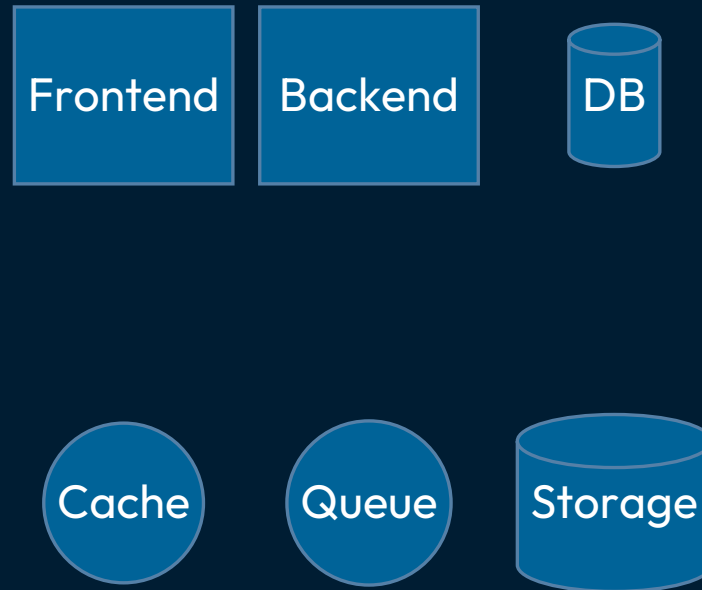


KEY INSIGHT

Inherits from core. The plot color palette can be overridden but per-series control is limited.

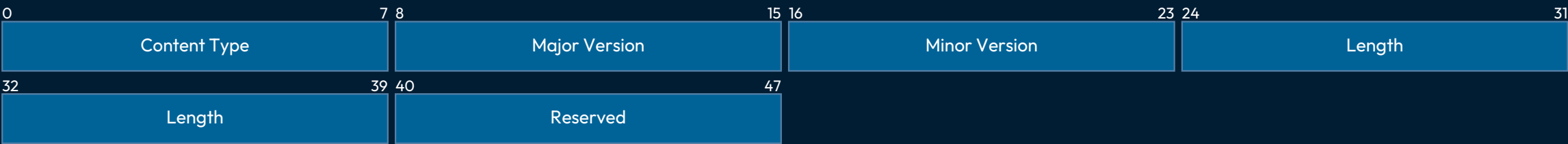
## 18 · BLOCK DIAGRAM 🔥

# Tile layout for system blocks.

**KEY INSIGHT**

Inherits from core. Spacing controlled by the `space:N` directive.

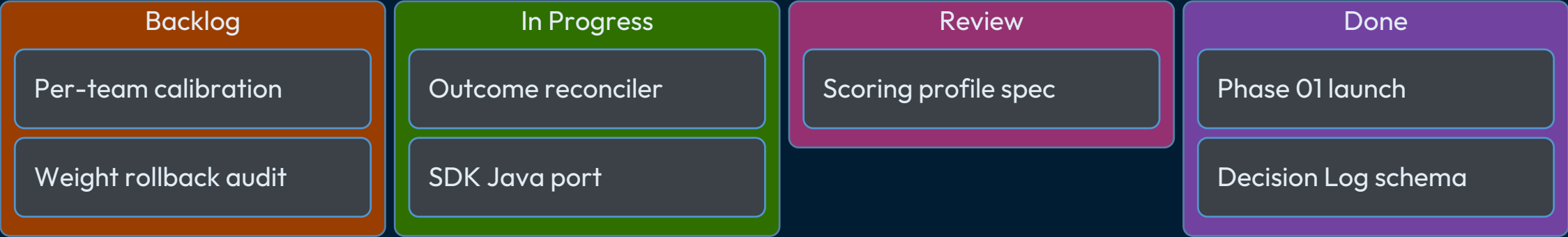
# Bit-level packet layout.



KEY INSIGHT

Inherits from core. Width-fixed at 32 bits per row by default; 0-N ranges define each field.

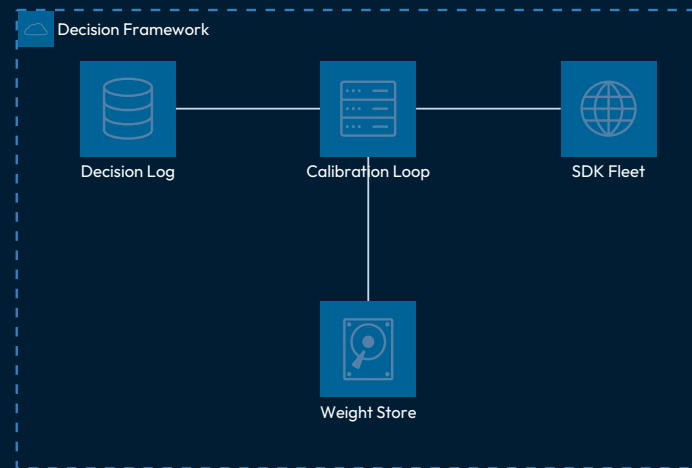
# A board with swim-lane columns.



KEY INSIGHT

Inherits from core. Per-card `[Title]@{ assigned: ..., priority: ... }` metadata supported but optional.

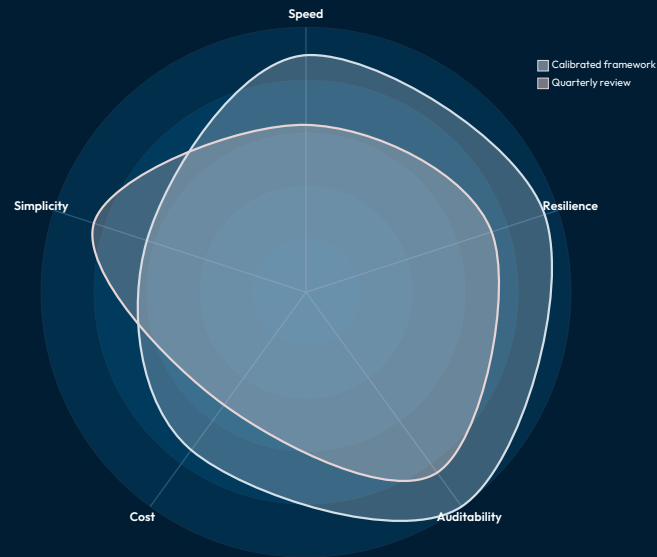
## Cloud-architecture style with services and groups.



### KEY INSIGHT

Inherits from core. Icons drawn from a registered set; default set is limited — register more via `mermaid.registerIconPacks`.

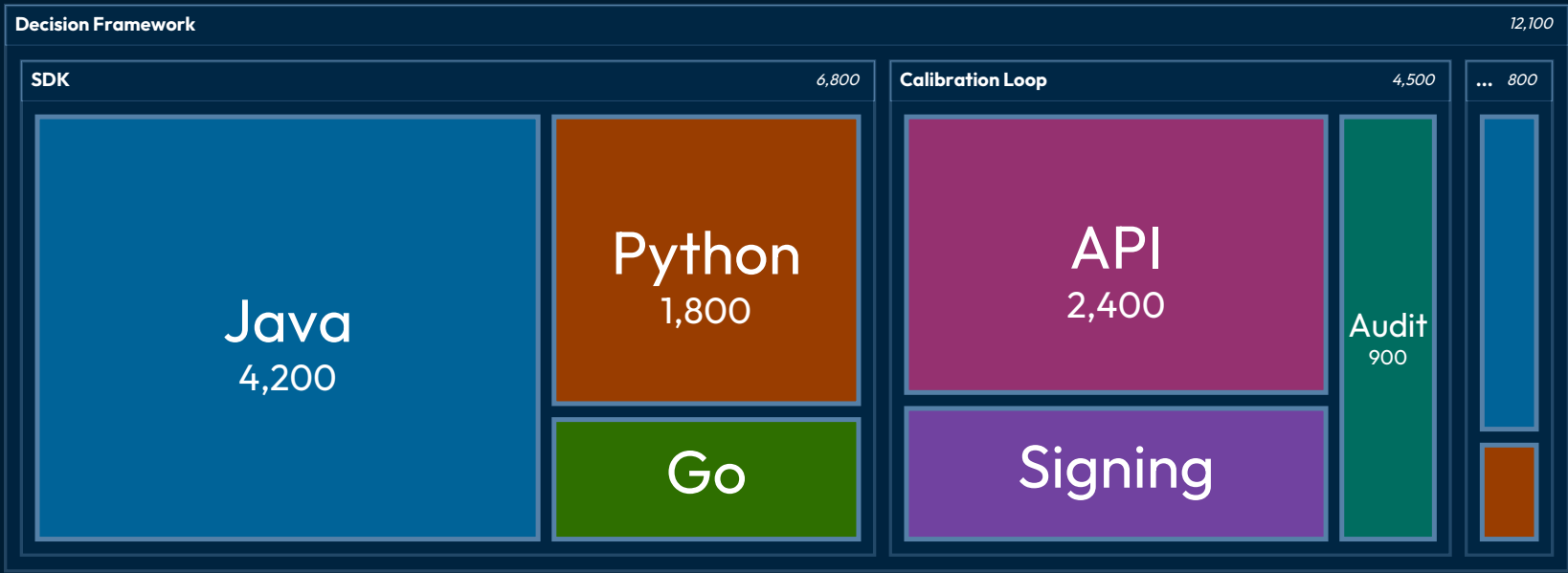
## Multi-axis polar chart.



### KEY INSIGHT

Inherits from core. Multiple `curve` lines plot on the same axes.

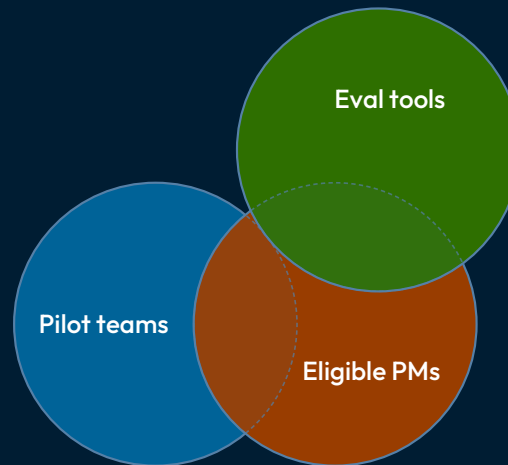
# Nested rectangles sized by value.



KEY INSIGHT

Inherits from core. Sized values cascade from leaves up; intermediate nodes don't take a value.

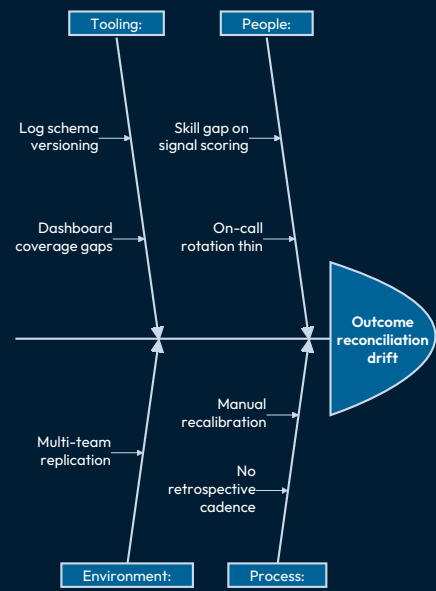
## Set overlap.



### KEY INSIGHT

Inherits from core. `union A, B` overlaps two sets; `set X["Label"]` gives a display name distinct from the identifier. Intersection labels (`union A, B["..."]`) are supported but crowd a three-set diagram, so this example leaves the regions unlabeled.

# Fishbone cause-and-effect.



KEY INSIGHT

Inherits from core. Branches default to four (people, process, tooling, environment) but any heading is allowed.

## Indented file-tree style hierarchy.

---



### KEY INSIGHT

Three named theme variables: `labelFontSize`, `labelColor`, `lineColor`. Indentation alone determines the tree.

MERMAID 11.14 · LATTICE THEME

# Twenty-six diagrams, one theme.

*Stable diagrams have rich theme control. Experimental diagrams marked 🔥 inherit from the core six variables and may evolve in syntax. Use this gallery as a reference for which diagram fits a given problem.*